Université de Caen Normandie

IUT Grand Ouest Normandie

Pôle de Caen

Bachelor Universitaire de Technologie

Science des Données

Projet Data Mining & Machine Learning

Système de Scoring Crédit

Automatisation de l'octroi de crédit bancaire

Romain LESUEUR & Matthias DEFRETIN

Année universitaire 2025/2026

Livrable final du projet

Résumé

Ce rapport présente le développement d'un système complet de scoring crédit, conçu pour automatiser la décision d'octroi de prêts bancaires. Le projet s'articule autour de trois phases majeures : la préparation des données (ETL) pour consolider et nettoyer des sources multiples, la modélisation (Machine Learning) visant à prédire le risque de défaut de paiement, et le développement d'une application web (Streamlit) destinée aux agents bancaires. Nous avons comparé plusieurs algorithmes (Régression Logistique, Random Forest, XGBoost) et traité le déséquilibre des classes via la technique SMOTE. Le modèle final, un XGBoost optimisé, a été intégré dans une interface interactive permettant de simuler l'éligibilité d'un client en temps réel.

Mots-clés : Scoring Crédit, Machine Learning, XGBoost, Python, Streamlit, ETL, Data Science.

Abstract

This report details the development of a comprehensive credit scoring system designed to automate bank loan approval decisions. The project is structured around three main phases : data preparation (ETL) to consolidate and clean multiple data sources, modeling (Machine Learning) to predict default risk, and the development of a web application (Streamlit) for bank agents. We compared several algorithms (Logistic Regression, Random Forest, XGBoost) and addressed class imbalance using the SMOTE technique. The final model, an optimized XGBoost, was integrated into an interactive interface allowing for real-time client eligibility simulation.

Keywords : Credit Scoring, Machine Learning, XGBoost, Python, Streamlit, ETL, Data Science.

SOMMAIRE

Introduction 1

1 Architecture et Organisation du Projet 2

1.1 Contexte et Objectifs 2

1.2 Organisation en Branches 2

1.3 Livrables 2

2 Phase 1 : Préparation des Données (ETL) 3

2.1 Fusion et Nettoyage 3

2.2 Feature Engineering 3

2.3 Optimisation 3

3 Phase 2 : Modélisation et Analyse 4

3.1 Analyse Exploratoire (EDA) 4

3.2 Stratégie de Modélisation 4

3.3 Résultats et Sélection 4

3.4 Simplification pour l'Application 4

4 Phase 3 : Application Streamlit 5

4.1 Fonctionnalités 5

4.2 Gestion des erreurs 5

Conclusion 6

Bibliographie 7

Introduction

L'octroi de crédit est une activité centrale pour les institutions bancaires, mais elle comporte un risque inhérent : le défaut de paiement. Pour les clients disposant d'un historique bancaire limité, l'évaluation de ce risque est particulièrement complexe. C'est dans ce contexte que s'inscrit notre projet "Système de Scoring Crédit".

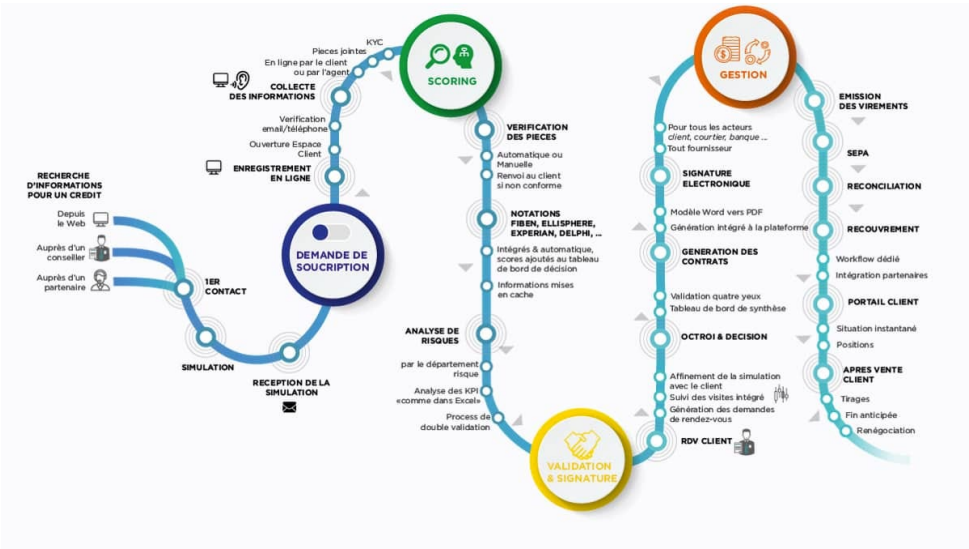


FIGURE 1 – Illustration du cycle de vie du crédit

L'objectif principal est de développer un outil d'aide à la décision capable de prédire la probabilité qu'un client rembourse son prêt, en se basant sur une multitude de données (personnelles, financières, historiques externes). Ce projet vise à automatiser le processus de décision pour gagner en efficacité tout en maîtrisant le risque financier.

Pour mener à bien cette mission, nous avons structuré notre approche en trois axes techniques complémentaires : l'ingénierie des données (ETL), la modélisation prédictive (Machine Learning) et le déploiement applicatif. Nous avons utilisé un jeu de données complexe nécessitant un important travail de nettoyage et de feature engineering avant de pouvoir entraîner des modèles performants.

Ce rapport retrace les différentes étapes du projet. Nous présenterons d'abord l'architecture globale et l'organisation de l'équipe. Nous détaillerons ensuite la phase cruciale de préparation des données, suivie de la phase de modélisation où nous justifierons nos choix algorithmiques. Enfin, nous présenterons l'application finale développée avec Streamlit, qui concrétise nos travaux sous la forme d'un outil utilisable par un conseiller bancaire.

1 Architecture et Organisation du Projet

1.1 Contexte et Objectifs

Le projet répond à une problématique de classification binaire supervisée. La variable cible (`TARGET`) indique si un client a eu des difficultés de paiement (1) ou non (0). Les objectifs spécifiques sont :

- **Prédire** le risque de défaut avec la meilleure précision possible (métrique ROC AUC).
- **Gérer** le fort déséquilibre des classes (environ 92% de bons payeurs pour 8% de défauts).
- **Fournir** une interface utilisateur simple pour l'exploitation du modèle.

1.2 Organisation en Branches

Pour assurer un développement structuré et collaboratif, nous avons divisé le projet en trois branches distinctes, correspondant aux phases logiques du cycle de vie de la donnée.

- **BRANCH 1 : ETL (Extract, Transform, Load)**
  Responsable : Matthias Defretin
  Cette phase concerne l'ingestion des données brutes, leur nettoyage, la gestion des valeurs manquantes et la création de nouvelles variables (Feature Engineering). C'est le socle du projet.
- **BRANCH 2 : Modeling**
  Responsables : Matthias Defretin & Romain Lesueur
  Cette phase couvre l'analyse exploratoire (EDA), l'entraînement des modèles, l'optimisation des hyperparamètres et la sélection du modèle final.
- **BRANCH 3 : Application**
  Responsable : Romain Lesueur
  Cette phase finale consiste à développer l'interface utilisateur avec Streamlit, intégrant le modèle entraîné pour réaliser des prédictions en temps réel.

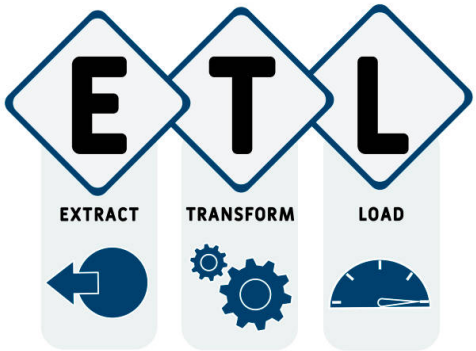
1.3 Livrables

Le projet aboutit à la production de plusieurs livrables concrets :

- Un ensemble de **Notebooks Jupyter** documentés (preprocessing, eda, modeling).
- Les fichiers de **modèles sérialisés** (.pkl) prêts à l'emploi.
- Une **application web** fonctionnelle (`app.py`).
- Ce **rapport technique** détaillant la méthodologie.

2 Phase 1 : Préparation des Données (ETL)

La qualité des données est déterminante pour la performance des modèles. Cette première phase a consisté à transformer des données brutes et dispersées en un dataset unique et cohérent.



2.1 Fusion et Nettoyage

Les données étaient réparties dans plusieurs fichiers (`application_train`, `previous_application`, etc.).

- **Fusion :** Nous avons réalisé des jointures (merges) pour regrouper les informations relatives à chaque client (`SK_ID_CURR`).
- **Valeurs manquantes :** Une stratégie d'imputation a été définie (médiane pour les variables numériques, mode pour les catégorielles) afin de ne pas perdre d'information précieuse.
- **Valeurs aberrantes :** Les outliers (ex : 365243 jours d'emploi) ont été traités ou remplacés par des valeurs nulles (NaN) pour éviter de biaiser le modèle.

2.2 Feature Engineering

Pour enrichir le modèle, nous avons créé de nouvelles variables "métier" pertinentes, telles que :

- **Ratios financiers :** Taux d'endettement, part de l'annuité dans le revenu, ratio crédit/prix du bien.
- **Agrégations :** Moyennes et sommes des crédits précédents (via les tables `bureau` et `previous_application`).

2.3 Optimisation

Le volume de données étant conséquent, nous avons optimisé les types de données (downcasting) et utilisé le Garbage Collector (`gc.collect()`) pour gérer la mémoire vive et éviter les erreurs lors des calculs.

3 Phase 2 : Modélisation et Analyse

3.1 Analyse Exploratoire (EDA)

Avant de modéliser, nous avons analysé la distribution des variables et leurs corrélations avec la cible.

— **Déséquilibre de classe :** Confirmé à 92% / 8%.
— **Corrélations :** Les variables `EXT_SOURCE` (scores externes) se sont révélées être les plus corrélées avec la cible. L'âge et l'ancienneté de l'emploi jouent également un rôle significatif.

3.2 Stratégie de Modélisation

Nous avons testé et comparé plusieurs algorithmes :

1. **Dummy Classifier :** Pour établir une baseline (score minimal à battre).
2. **Régression Logistique :** Pour sa simplicité et son interprétabilité.
3. **Random Forest :** Pour sa robustesse et sa capacité à gérer les relations non linéaires.
4. **XGBoost :** Pour sa performance reconnue sur les données tabulaires structurées.

Pour traiter le déséquilibre, nous avons utilisé la technique SMOTE (Synthetic Minority Over-sampling Technique) dans nos pipelines d'entraînement, ainsi que l'ajustement des poids des classes (`scale_pos_weight`).

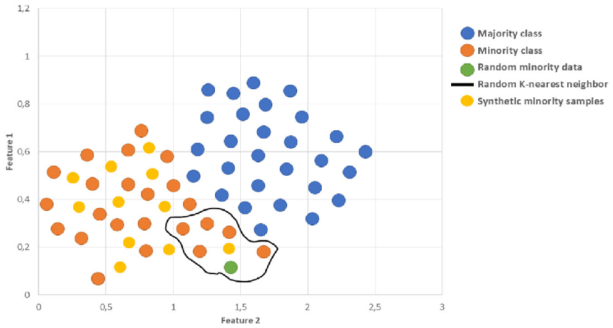


FIGURE 2 – Illustration de la technique SMOTE

3.3 Résultats et Sélection

Le modèle XGBoost a obtenu les meilleures performances en termes d'AUC (Area Under the Curve). C'est ce modèle qui a été retenu pour l'application finale.

3.4 Simplification pour l'Application

Le modèle complet utilisait plus de 100 variables, ce qui rendait la saisie manuelle impossible pour un utilisateur humain. Nous avons donc : 1. Analysé l'importance des features (Feature Importance). 2. Sélectionné les 15 variables les plus influentes (Scores externes, Annuité, Âge, etc.). 3. Ré-entraîné un modèle "léger" (`xgb_small`) sur ces seules 15 variables.

Cette étape cruciale a permis de concilier performance prédictive et utilisabilité de l'application.

4 Phase 3 : Application Streamlit

L'objectif de cette phase était de rendre le modèle accessible via une interface web conviviale. Nous avons choisi la librairie Streamlit pour sa rapidité de développement en Python.

4.1 Fonctionnalités

L'application (`app.py`) propose les fonctionnalités suivantes :

- **Formulaire de saisie :** L'utilisateur (conseiller bancaire) saisit les informations du nouveau client. Le formulaire est organisé en sections logiques (Informations personnelles, Données financières, Scores).
- **Chargement du modèle :** L'application charge dynamiquement le modèle simplifié (`best_model_xgb_small.pkl`) et la liste des features.
- **Prédiction en temps réel :** Au clic sur "Lancer la Prédiction", le modèle calcule la probabilité de défaut.
- **Aide à la décision :** Le résultat est affiché clairement avec un code couleur (Vert = Crédit Accepté, Rouge = Risque Élevé) et une jauge de probabilité.

4.2 Gestion des erreurs

L'application gère les cas où certaines variables techniques (non saisies dans le formulaire) seraient attendues par le modèle, en leur attribuant des valeurs par défaut neutres (0) pour garantir l'exécution sans crash.

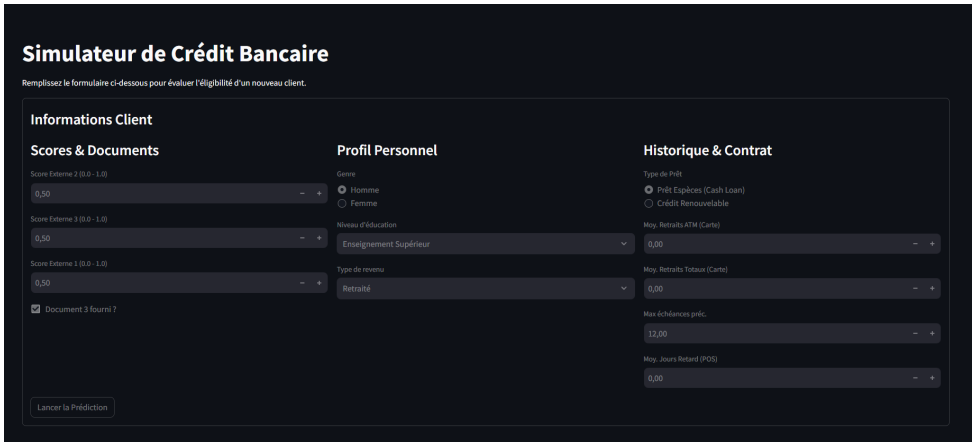


FIGURE 3 – Aperçu de l'interface de simulation (Formulaire de saisie)

Conclusion

Ce projet de "Système de Scoring Crédit" a constitué une expérience enrichissante, nous permettant de mettre en œuvre l'intégralité de la chaîne de valeur de la donnée, depuis l'extraction brute jusqu'à la livraison d'un outil décisionnel opérationnel. Il a nécessité la mobilisation de compétences variées, allant de l'ingénierie des données à la modélisation statistique, en passant par le développement logiciel.

Sur le plan technique, nous avons réussi à surmonter les défis inhérents à la volumétrie importante des données et au fort déséquilibre des classes cibles. L'application d'une méthodologie rigoureuse, incluant un nettoyage approfondi (ETL optimisé) et l'utilisation de techniques de rééquilibrage comme SMOTE, a été déterminante pour la qualité de nos résultats. Le choix stratégique de simplifier le modèle final pour ne conserver que les 15 variables les plus pertinentes s'est avéré être un compromis judicieux. Cette approche a permis de concilier la performance prédictive (avec un AUC satisfaisant) et l'opérabilité de l'application, rendant la saisie manuelle par un conseiller bancaire réaliste.

Pour aller plus loin, plusieurs pistes d'amélioration sont envisageables. D'une part, l'intégration de méthodes d'explicabilité locale, telles que les valeurs SHAP, permettrait au conseiller de comprendre précisément pourquoi un crédit est refusé (ex : "taux d'endettement trop élevé" ou "historique insuffisant"), renforçant ainsi la confiance dans l'outil. D'autre part, le modèle pourrait être enrichi par de nouvelles sources de données (Open Banking, comportement transactionnel) pour affiner encore la précision du scoring. Enfin, une mise en production via une API REST permettrait d'intégrer ce moteur de décision directement dans le système d'information global d'une banque.

Bibliographie

Documentation Technique

— **Scikit-learn Documentation** : https://scikit-learn.org/stable/
— **XGBoost Documentation** : https://xgboost.readthedocs.io/
— **Streamlit Documentation** : https://docs.streamlit.io/
— **Pandas User Guide** : https://pandas.pydata.org/docs/user_guide/

Sources Académiques

— Supports de cours "Data Mining & Machine Learning", IUT Grand Ouest Normandie.
— "Handling Imbalanced Datasets with SMOTE", Journal of Artificial Intelligence Research.